DOCUMENTO TÉCNICO

naPorta

API de gestão de pedidos de entrega e portal web de operação. Monorepo com NestJS, Prisma e PostgreSQL, autenticação JWT, exclusão lógica, cobertura de testes com piso configurado e imagens Docker de produção.

10	153	6	8.300
ENDPOINTS REST	TESTES AUTOMATIZADOS	TABELAS · 19 ÍNDICES	LINHAS DE CÓDIGO

STACK

Node.js 24 (ESM) · TypeScript 6.0 · NestJS 12 · Prisma 7 · PostgreSQL 16 · Vitest 5 · ESLint 10 · Next.js 16 · React 19 · Tailwind 4 · Docker

Versão 0.1.0 · 22 de outubro de 2026 · Repositório local, sem publicação em remoto

— Sumário

1. Resumo executivo
2. Contexto e escopo
3. Arquitetura do sistema
4. Stack técnica
5. Modelo de dados
6. A API
7. Autenticação e segurança
8. O portal web
9. Testes e qualidade
10. Docker e operação
11. Decisões técnicas
12. Limitações conhecidas e próximos passos
13. Anexo: referência de comandos

1. Resumo executivo

A naPorta é uma API de gestão de pedidos de entrega acompanhada de um portal web de operação. O pedido é a entidade central: nasce com um número de negócio sequencial, tem fluxo de status controlado, endereço de entrega, itens com preço exato e é removido por exclusão lógica — a linha permanece no banco, mas some das leituras.

1.1 O que o sistema faz

- **Autenticação** por JWT com access token curto e refresh token rotativo; detecção de reuso de token revoga todas as sessões do usuário.
- **CRUD de pedidos** com listagem filtrável (número, status, período), paginação, ordenação com lista branca e transições de status validadas.
- **Propriedade** por dono do pedido: o usuário comum só acessa os próprios pedidos; o administrador acessa todos.
- **Portal web** que consome a API real, com sessão em cookie `httpOnly` e renovação transparente do token.
- **Contrato compartilhado** entre os dois apps: se a API mudar o formato das respostas, o portal deixa de compilar.

1.2 Números

INDICADOR	VALOR	DETALHE
Endpoints	10	versionados em `/api/v1`
Telas do portal	4	login, dashboard, novo pedido, detalhe
Testes	153	64 unitários + 89 e2e
Cobertura	78% / 85% / 79%	statements / functions / lines (pisos 75 / 80 / 75)
Modelo de dados	6 tabelas	19 índices, 1 migration versionada
Código	8.219 linhas	API 3.883 · testes 1.790 · portal 1.709 · contratos 263
Imagens	175 MB + 97 MB	API e portal, usuário sem privilégios, health check

2. Contexto e escopo

O sistema nasce de um desafio técnico de backend. O escopo pedido — API REST, JWT, banco relacional, ORM, seed, README, testes e Docker — foi entregue integralmente, e acrescido de três bônus: portal web, documentação OpenAPI gerada do código e contrato compartilhado.

2.1 Requisitos do desafio × entrega

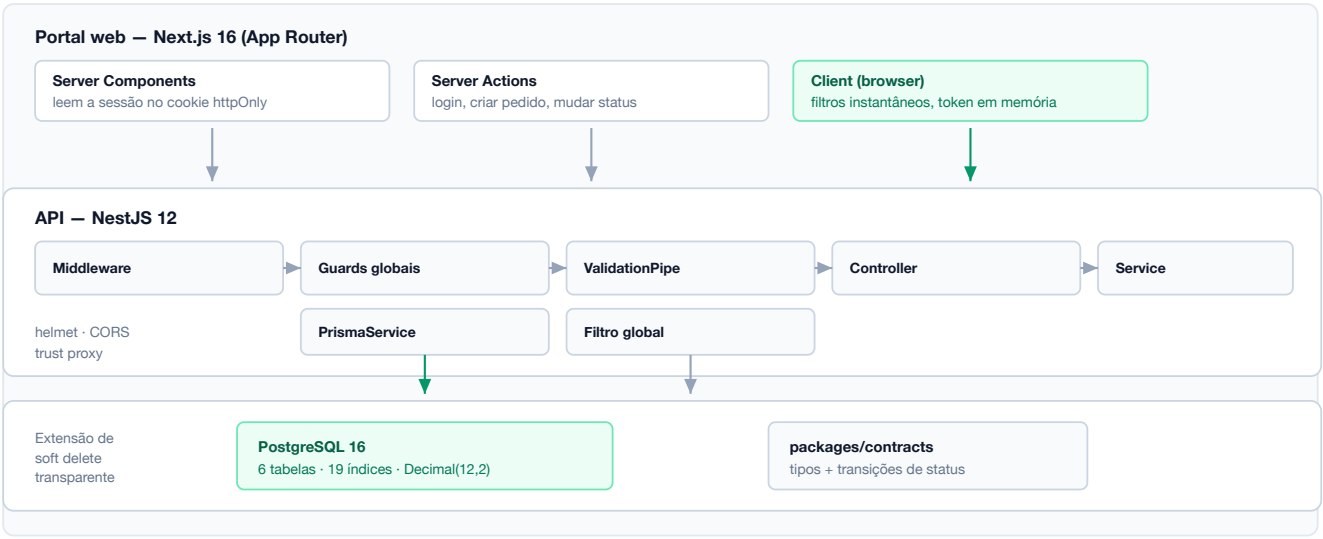
REQUISITO	STATUS	ONDE
API em Node.js com NestJS	ENTREGUE	`apps/api`
Autenticação JWT com Bearer	ENTREGUE	`modules/auth`
Banco relacional	ENTREGUE	PostgreSQL 16 (Docker)
ORM	ENTREGUE	Prisma 7
Rotas REST	ENTREGUE	`/api/v1`
README com execução e scripts	ENTREGUE	`README.md` + `apps/web/README.md`
Seed com pedidos fictícios	ENTREGUE	`prisma/seed.ts` (somente por comando)
Criar, listar, filtrar, editar, excluir	ENTREGUE	`modules/orders`
Exclusão lógica	ENTREGUE	extensão do Prisma
Clean Code	ENTREGUE	módulos, guards, filtros, mappers, tipagem estrita
Testes automatizados	ENTREGUE	153 testes + cobertura com piso
Docker	ENTREGUE	4 serviços na stack
Linter	ENTREGUE	ESLint 10 com type-checking
Serverless	FORA	decisão consciente: contêiner de longa duração é mais simples de operar e testar
Bônus: portal web	ENTREGUE	`apps/web` (Next.js 16)
Bônus: documentação da API	ENTREGUE	OpenAPI gerado do código em `/docs`
Bônus: contrato compartilhado	ENTREGUE	`packages/contracts`
Bônus: CI/CD	ADIADO	os scripts já existem na raiz do monorepo

2.2 Princípio de dados

> **O banco nasce vazio — e continua assim até alguém pedir.**
>
> A aplicação **nunca** popula o banco sozinha. O serviço de migrations executa apenas `prisma migrate deploy`; os dados de demonstração entram só por comando explícito (`npm run seed` ou `npm run seed:api`). A suíte de testes ainda se recusa a rodar contra um banco que não seja de teste — os testes fazem `TRUNCATE`, e apagar dados de trabalho por acidente seria inaceitável.

3. Arquitetura do sistema

Monorepo com npm workspaces: duas aplicações e um pacote compartilhado. A API e o portal conversam por HTTP, e o contrato entre eles é um workspace compilado — não uma documentação que pode envelhecer.



Fluxo de uma requisição: o portal fala com a API por dois caminhos — no servidor (cookie httpOnly) e no browser (token em memória) — sempre atravessando o mesmo pipeline de segurança.

3.1 Anatomia de uma requisição

1. **Middleware** — `helmet` (CSP, HSTS, sniffing), CORS por whitelist e `trust proxy`.
2. **Guards globais** — nesta ordem: rate limit, autenticação (Bearer) e autorização por papel. Rotas sem `@Public()` exigem token.
3. **ValidationPipe** — valida os DTOs, remove campos não declarados (`whitelist`) e rejeita campos desconhecidos (`forbidNonWhitelisted`) com **422**.
4. **Controller** — apenas HTTP: parsing e delegação.
5. **Service** — regras de negócio, autorização por propriedade e transações.
6. **Prisma** — acesso a dados com a extensão de soft delete aplicada.
7. **Filtro global** — qualquer erro vira o mesmo formato de resposta.

3.2 Organização do código

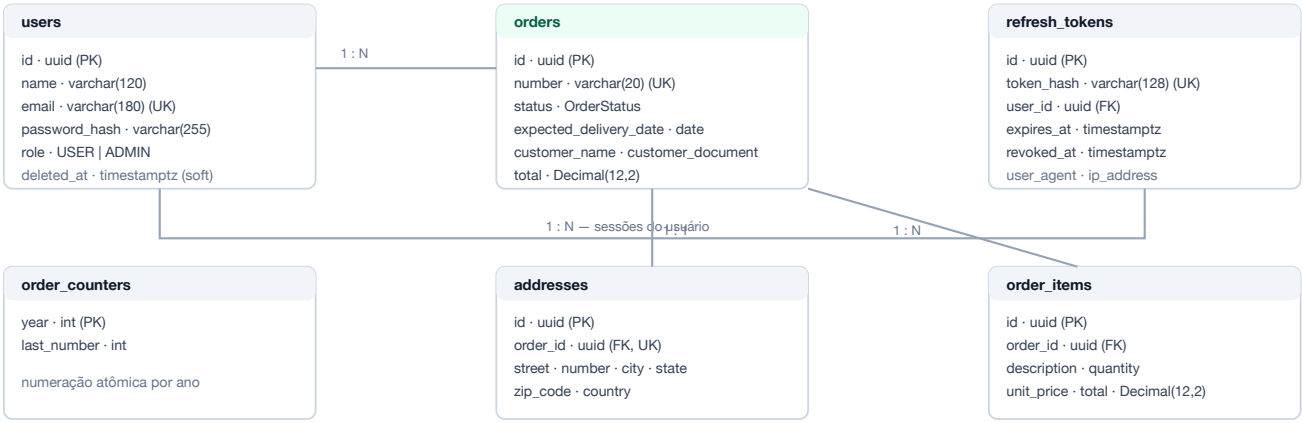
apps/api/src/common	decorators, mapeamento de erros, filtro global de exceções, limitadores, utilitários (preço, versão) e validadores (CPF/CNPJ)
apps/api/src/config	ambiente tipado, validação com Joi e namespaces de configuração
apps/api/src/modules	`auth` (autenticação, guards, senhas, tokens) · `orders` (CRUD, filtros, status, numeração) · `users` (administração) · `health` (liveness e readiness) · `prisma` (client e extensão de soft delete)
apps/web/src/app	rotas do App Router e Server Actions
apps/web/src/components	interface: tabela, filtros, formulários, cabeçalho e provedor de sessão

apps/web/src/lib	client da API (servidor e browser), leitura de cookies e formatação
packages/contracts	contrato de dados e regras de transição de status compartilhados

4. Stack técnica

CAMADA	TECNOLOGIA	OBSERVAÇÕES
Runtime	Node.js 24 (ESM)	Projeto em `"type": "module"`: o ecossistema NestJS 12 é ESM-first e forçar CommonJS exigiria pacotes incompatíveis.
Linguagem	TypeScript 6.0 `strict`	`nodenext`, `noUncheckedIndexedAccess`, resolução de tipos via `import type` — exceto nas classes injetadas, que o Nest resolve em runtime.
Framework	NestJS 12	Módulos, guards globais, pipes, filtros e interceptores de log.
ORM	Prisma 7	Client gerado como TypeScript (`src/generated/prisma`), sem engine Rust: cold start mais rápido e os tipos no repositório.
Banco	PostgreSQL 16	`Decimal(12,2)` para dinheiro, `UUID` v4 para chaves, cascade nos filhos.
Testes	Vitest 5 + Supertest	`unplugin-swc` no lugar do esbuild — o Nest depende de `emitDecoratorMetadata`, que o esbuild não implementa.
Qualidade	ESLint 10 + Prettier 3	Flat config com `projectService`: regras que dependem de tipo (`no-floating-promises`, `no-unsafe-assignment`) valem de verdade.
Logs	nestjs-pino	JSON no container, legível no desenvolvimento; cabeçalhos sensíveis censurados (`authorization`, `cookie`).
Documentação	OpenAPI 3 (Swagger UI)	Schemas gerados do código pelo plugin do `@nestjs/swagger`: os DTOs são a fonte da verdade da validação e da documentação.
Portal	Next.js 16 + React 19 + Tailwind 4	App Router com Server Components, Server Actions e `output: standalone`.
Infraestrutura	Docker (Compose)	Imagens multi-stage, usuário sem privilégios, `tini` como PID 1 e health check.

5. Modelo de dados



Seis tabelas, 19 índices. Endereço e itens são tabelas próprias (não campos JSON): a integridade referencial fica no banco.

5.1 Entidades

TABELA	PAPEL	OBSERVAÇÕES
`users`	Usuários da API	Senha só como hash argon2id; papel `USER` / `ADMIN`; `deleted_at` para exclusão lógica.
`orders`	Pedidos	`number` único **global** (nunca reutilizado, nem após exclusão lógica); `total` desnormalizado para listagem sem agregação.
`addresses`	Endereço de entrega	Relação 1:1 com o pedido, com cascade do banco.
`order_items`	Itens do pedido	Relação 1:N; `unit_price` e `total` em `Decimal(12,2)`.
`refresh_tokens`	Sessões ativas	Guarda o **hash SHA-256** do token — nunca o token. `revoked_at` implementa a rotação.
`order_counters`	Sequencial anual	Permite o upsert atômico que gera o número do pedido.

5.2 Número do pedido

O número segue o padrão `NP-AAAA-NNNNN` (ex.: `NP-2026-00001`) e é atribuído por ano com um *upsert* atômico — a atomicidade vem do próprio banco, sem lock explícito na aplicação:

```
INSERT INTO order_counters (year, last_number) VALUES ($1, 1)
ON CONFLICT (year) DO UPDATE SET last_number = order_counters.last_number + 1
RETURNING last_number;
```

Duas requisições simultâneas nunca recebem o mesmo número — a suíte cobre isso com 10 criações concorrentes e 25 chamadas diretas ao gerador.

5.3 Dinheiro

Valores monetários usam `Decimal(12,2)` no banco e são serializados como `number` apenas na borda da resposta. A entrada aceita número ou string, com no máximo duas casas decimais: valor com mais casas é **rejeitado**, nunca arredondado em silêncio. O efeito aparece no teste: `3 × 0,10 + 0,20 = 0,50` — e não `0,5000000000000001`.

6. A API

Convenção REST com versionamento por URI (`/api/v1`). Todos os endpoints exigem `Authorization: Bearer <accessToken>`, exceto os marcados como públicos.

6.1 Endpoints

MÉTODO	ROTA	PÚBLICO	DESCRIÇÃO
POST	`/api/v1/auth/register`	sim	Cria a conta (papel `USER`) e devolve a sessão inicial.
POST	`/api/v1/auth/login`	sim	Autentica e emite access + refresh token.
POST	`/api/v1/auth/refresh`	sim	Renova a sessão e rotaciona o refresh token.
POST	`/api/v1/auth/logout`	não	Revoga o refresh token informado. Responde `204`.
GET	`/api/v1/auth/me`	não	Dados do usuário autenticado.
POST	`/api/v1/orders`	não	Cria um pedido. Número e total são definidos pelo servidor.
GET	`/api/v1/orders`	não	Lista com filtros, paginação e ordenação.
GET	`/api/v1/orders/:id`	não	Detalha um pedido.
PATCH	`/api/v1/orders/:id`	não	Atualiza campos e/ou o status.
DELETE	`/api/v1/orders/:id`	não	Exclusão lógica. Responde `204`.
GET	`/api/v1/users`	não	Lista usuários — restrito a `ADMIN`.
GET	`/api/v1/health`	sim	Liveness: o processo está vivo?
GET	`/api/v1/health/ready`	sim	Readiness: o banco responde?

6.2 Filtros e paginação

PARÂMETRO	EXEMPLO	DESCRIÇÃO
`number`	`NP-2026-00001`	Número do pedido (comparação exata).
`status`	`PENDING`	`PENDING` · `CONFIRMED` · `PROCESSING` · `SHIPPED` · `DELIVERED` · `CANCELLED`
`startDate` / `endDate`	`2026-01-01`	Período sobre a **data prevista de entrega**.
`page` / `limit`	`1` / `20`	Paginação; limite máximo de 100.
`sortBy` / `order`	`total` / `desc`	Ordenação por lista branca: `createdAt`, `expectedDeliveryDate`, `total`, `number`, `status`.

FORMATO DA RESPOSTA PAGINADA

```
{
  "data": [ /* pedidos */ ],
  "meta": { "total": 60, "page": 1, "limit": 20, "totalPages": 3 }
}
```

6.3 Formato de erro

Todo erro — validação, negócio, autorização ou infraestrutura — sai no mesmo formato:

```
{
  "statusCode": 409,
  "error": "Conflict",
  "message": "Não é possível alterar o status de PENDING para DELIVERED.",
  "path": "/api/v1/orders/6f1c…",
  "timestamp": "2026-10-22T05:02:22.091Z",
  "requestId": "42"
}
```

ERROS CONHECIDOS DO BANCO SÃO TRADUZIDOS

ORIGEM	HTTP	MOTIVO
`P2002`	409	Violação de unicidade.
`P2025`	404	Registro não encontrado.
`P2003`	400	Referência inválida (foreign key).
Códigos de conexão	503	Banco indisponível.

6.4 Regras de negócio

- **Total** — sempre calculado no servidor a partir dos itens. O cliente não envia o campo; enviá-lo é rejeitado com `422`.
- **Status** — o fluxo é `PENDING → CONFIRMED → PROCESSING → SHIPPED → DELIVERED`; `CANCELLED` é alcançável de qualquer etapa não terminal. Pular etapa ou voltar de um status final devolve `409`.
- **Documento** — CPF e CNPJ validados pelos dígitos verificadores; gravado apenas com dígitos.
- **Data prevista** — formato `YYYY-MM-DD`, válida e não no passado.
- **Propriedade** — usuário comum só acessa os próprios pedidos; pedido de outra pessoa responde `404` (e não `403`): a API não confirma a existência de registros que o solicitante não pode ver.

7. Autenticação e segurança

7.1 O ciclo da sessão

ETAPA	ACONTECE	GARANTIA
Login	A API valida a senha (argon2id) e emite um access token JWT (15 min) e um refresh token opaco (7 dias).	O refresh token é gravado apenas como **hash SHA-256**. Vazamento do banco não permite reaproveitar sessão.
Uso	O access token vai no header `Authorization: Bearer`; a API recarrega o usuário do banco a cada requisição.	Conta desativada ou excluída perde o acesso **imediatamente**, sem esperar o token expirar; o papel usado na autorização está sempre atualizado.
Renovação	O refresh token é Presentation trocado por um novo par; o antigo é revogado.	Reapresentar um token já rotacionado é tratado como **roubo**: todas as sessões do usuário são revogadas.
Logout	O refresh token é revogado e os cookies são limpos.	A sessão não pode ser retomada.

7.2 Medidas aplicadas

MEDIDA	IMPLEMENTAÇÃO	POR QUE
Hash de senha	argon2id — 19 MiB, 2 iterações, 1 thread (OWASP)	Resistente a GPU/ASIC; serviço isolado para permitir troca de algoritmo.
Política de senha	Mínimo 8 caracteres, ao menos uma letra e um número	Rejeitada com `422` antes de qualquer escrita.
Anti enumeração de contas	Mensagem idêntica e verificação de um hash descartável	O tempo de resposta não revela quais e-mails existem — nem quando o e-mail não existe.
Anti escalonamento	`role` não existe no DTO de cadastro	Todo cadastro nasce `USER`; enviar o papel é `422`.
Guards globais	Rate limit → autenticação → autorização por papel	Política *secure by default*: rota nova já nasce protegida.
Rate limit	5/min em credenciais, 100/min global, configurável	Força bruta; resposta `429` com `Retry-After`.
Cabeçalhos	`helmet` com CSP, HSTS e anti-sniffing	Reduz a superfície de XSS e clickjacking.
CORS	Whitelist de origens	Só o portal e as origens declaradas acessam a API.
IP do cliente	`TRUST_PROXY=0` por padrão	`X-Forwarded-For` é forjável; confiar nele por padrão permitiria escapar do rate limit. Só há proxy reverso real quando há proxy reverso real.
Segredos	Validados no boot; em produção, obrigatórios e distintos	Segredos de desenvolvimento são recusados em produção.
Logs	Censura de `authorization` e `cookie`	Token não vaza para o agregador de logs.

Uma armadilha que custou tempo: `import type` quebra a injeção do Nest.

O Nest resolve dependências pelo tipo emitido em `design:paramtypes`, que precisa do **valor** da classe em runtime. Com `import type`, o Nest vê `undefined` e falha com "*can't resolve dependencies*". Por isso as classes injetadas são importadas como valor, com comentário explicando o motivo.

8. O portal web

Portal de operação em Next.js 16 (App Router). Existe porque uma API que ninguém consegue usar de verdade não comprova nada — e o enunciado convida a ir além do escopo.

8.1 Telas

ROTA	TELA	CONTEÚDO
`/login`	Login	E-mail e senha; erro exibido sem recarregar a página.
`/`	Dashboard	Lista de pedidos com filtros (número, status, período), paginação e total; os filtros respondem sem recarregar a página.
`/orders/new`	Novo pedido	Cliente, documento, endereço e itens (lista dinâmica com preço e quantidade).
`/orders/[id]`	Detalhe	Itens, endereço, cliente, ações de status disponíveis e exclusão (com confirmação).

8.2 Como o portal trata a sessão

Access token	Vive **apenas em memória** no cliente (via contexto React). Nunca vai para `localStorage`, que persiste entre abas e é legível por qualquer script injetado.
Refresh token	Fica **sempre** no cookie `httpOnly`: o JavaScript do browser nunca o vê. A rotação acontece no servidor.
Cookies	`httpOnly` + `sameSite=lax` + `secure` em produção.
Renovação	Um `401` dispara a Server Action de renovação (que usa o refresh token) e a requisição é repetida — transparente para o usuário.
CORS	Como o browser fala diretamente com a API nos filtros instantâneos, a origem do portal precisa estar na lista `CORS_ORIGINS` da API.

8.3 Tratamento de erro no portal

A página de detalhe tem duas camadas de proteção: um identificador inválido (a API recusa com `400`) é tratado como *pedido inexistente* e leva à página de "não encontrado" (`404`); qualquer outra falha da API cai na fronteira de erro da rota (`error.tsx`), que explica o que houve e oferece "tentar de novo" e "voltar para os pedidos" — em vez de um 500 sem contexto.

8.4 O contrato compartilhado

O portal e a API conversam por HTTP, mas os *tipos* vêm do mesmo lugar: `packages/contracts`. As classes de resposta da API implementam esses tipos — se a API mudar o formato de uma resposta, o **portal deixa de compilar**. As regras de transição de status também moram lá: a API valida e o portal oferece as ações, e os dois concordam com o fluxo.

```
// apps/api — a resposta implementa o contrato
export class OrderResponse implements OrderContract {
  id!: string;
  number!: string;
  status!: OrderStatus;
  /* ... */
}
```

9. Testes e qualidade

153 testes em duas suítes com finalidades distintas, e um piso de cobertura que faz a suíte falhar se a qualidade cair.

9.1 As duas suítes

SUÍTE	COMANDO	BANCO	COBRE
Unitária	`npm test`	não precisa	Funções puras: transições de status, CPF/CNPJ, precisão decimal, mapeamento de erros do Prisma, parsing de duração, configuração do limitador e hash de senha.
E2E / integração	`npm run test:e2e`	`naporta_test`	A aplicação inteira via HTTP: autenticação, refresh rotativo, CRUD, filtros, paginação, propriedade, soft delete, seed, rate limit e concorrência.

9.2 Cobertura

MÉTRICA	ATUAL	PISO	OBSERVAÇÃO
Statements	78%	75%	O piso é configurado no Vitest: se a cobertura cair, `npm run test:e2e:cov` **falha**. A suíte é rede de segurança, não formulário.
Functions	85%	80%	
Lines	79%	75%	

9.3 O que a suíte prova (e não só "passa")

CENÁRIO	COMO É VERIFICADO
Soft delete é real	A linha continua na tabela com `deleted_at` preenchido e some das leituras.
O numerador não colide	10 criações de pedido **simultâneas** → 10 números distintos.
Dinheiro é exato	`3 × 0,10 + 0,20 = 0,50` — nunca `0,5000000000000001`.
Reuso de token é detectado	Reapresentar um refresh token rotacionado revoga todas as sessões do usuário.
Conta inativa perde o acesso	Token válido deixa de valer imediatamente após a desativação.
Rate limit existe	Acima do limite: `429` com `Retry-After` e corpo no contrato.
Erro tem contrato único	Quatro tipos de falha devolvem o mesmo formato.
Propriedade é respeitada	Pedido de outra pessoa → `404` em leitura, edição e exclusão.
A suíte não toca o banco de dev	O setup global **recusa** rodar contra um banco que não seja de teste (os testes fazem `TRUNCATE`).

9.4 Qualidade estática

O lint roda com informação de tipo (`projectService`), o que habilita regras que só fazem sentido com o compilador: `no-floating-promises`, `no-unsafe-assignment`, `require-await`. A formatação é responsibility do Prettier, aplicado como regra do próprio ESLint — uma fonte única, sem disputa entre as duas ferramentas.

10. Docker e operação

10.1 A stack

SERVIÇO	PAPEL	DETALHE
`db`	PostgreSQL 16 (dev)	Volume nomeado, health check com `pg_isready`.
`db-test`	PostgreSQL 16 (testes)	Banco isolado, atrás do profile `test`.
`migrate`	Aplica migrations e **encerra**	Usa o estágio `build` (com o CLI do Prisma). A API só sobe com `service_completed_successfully`.
`api`	Web API	175 MB, usuário `node` sem privilégios, `tini` como PID 1, health check em `/api/v1/health`.
`web`	Portal	97 MB, saída `standalone` do Next, usuário sem privilégios, health check em `/login`.
`seed`	Popula o banco com dados fictícios	Atrás do profile `seed`: `docker compose up` continua subindo o banco **vazio**. Usa a imagem de build — a imagem final da API não carrega o CLI do Prisma.

10.2 A imagem da API

Quatro estágios, cada um com um propósito — e o resultado final sem nada que só serve para construir:

```
deps      → npm ci (camada cacheável)
build     → prisma generate + compilação dos contratos e da API
prod-deps → npm ci --omit=dev (só dependências de produção)
runtime   → imagem final: dist + node_modules enxuto, usuário node
```

O CLI do Prisma, o Prisma Studio e os dev tools (≈190 MB) são removidos do runtime: as migrations rodam no serviço separado `migrate`.

10.3 Integração contínua

O workflow `.github/workflows/ci.yml` roda a cada push e pull request, em três jobs paralelos:

JOB	PASSOS	OBSERVAÇÃO
qualidade	`npm ci` → Prisma Client → contratos → `lint` → `format:check` → `typecheck` → `build`	O type-check depende dos tipos compilados do pacote de contratos.
testes	Postgres 16 de serviço → suíte unitária → suíte e2e → cobertura (com piso) → artefato	O banco de testes é isolado; o de desenvolvimento nunca é tocado.
docker	Build das imagens da API e do portal	Valida os Dockerfiles a cada push.

O banco do GitHub Actions é um *serviço* do runner: as migrations são aplicadas pelo setup global da suíte, e o ambiente de teste se recusa a rodar contra um banco que não seja de teste.

10.4 Operação

- **Configuração validada no boot** — variáveis obrigatórias ausentes ou inválidas derrubam a aplicação na inicialização, com mensagem explícita.
- **Encerramento gracioso** — `tini` encaminha o `SIGTERM` e o Nest fecha o pool do banco antes de sair.
- **Health checks distintos** — `/health` (liveness) não depende do banco: se o banco cair, a API continua viva e o orquestradorAge só no banco. Já `/health/ready` verifica o banco e devolve `503`.
- **Logs estruturados** — JSON no container, legível no desenvolvimento, com campos sensíveis censurados.

11. Decisões técnicas

As escolhas que mais definem o sistema — e o motivo de cada uma.

TEMA	DECISÃO	POR QUÊ
Módulos ESM	Projeto em ESM	O ecossistema NestJS 12 é ESM-first; forçar CommonJS exigiria pacotes incompatíveis.
TypeScript	6.0 com `nodenext`	Única versão que satisfaz os peers de `@nestjs/schematics` e do `typescript-eslint`.
Prisma	Client gerado como TypeScript, sem engine Rust	Cold start mais rápido, tipos no repositório, sem download de engine.
Prisma + herança	Composição (`PrismaService.client`)	A extensão devolve um objeto novo; herdar exigiria `as unknown as` e perderia a tipagem.
Testes	Vitest + `unplugin-swc`	O Nest depende de `emitDecoratorMetadata`, que o esbuild não implementa.
Soft delete	Extensão do Prisma	Registro excluído nunca vaza por uma leitura "esquecida" de filtro.
Número do pedido	Upsert atômico no Postgres	Atomicidade vem do banco — sem lock explícito na aplicação.
Unicidade do número	Global (nunca reutilizado)	É identificador de negócio, como nota fiscal: reuso criaria ambiguidade na auditoria.
Dinheiro	`Decimal(12,2)` no banco	Precisão exata na soma; `number` só na borda da resposta.
Total do pedido	Sempre calculado no servidor	O cliente não informa o valor — e o campo é rejeitado se enviado.
Senhas	argon2id (parâmetros OWASP)	Resistente a GPU/ASIC; serviço isolado para permitir troca de algoritmo.
Refresh token	Valor opaco + hash SHA-256 persistido	Vazamento do banco não permite reaproveitar nenhuma sessão.
Reuso de token	Revoga todas as sessões do usuário	Indício de roubo: invalida tudo, não só o token.
Erros	Filtro global + contrato único	Nenhuma rota devolve um formato diferente de erro.
Validação	`422 Unprocessable Entity`	Distingue "requisição malformada" (400) de "regra de negócio violada" (422).
Propriedade	Pedido alheio → 404	A API não confirma a existência de registros que o solicitante não pode ver.
`TRUST_PROXY`	`0` por padrão	`X-Forwarded-For` forjável escaparia do rate limit.
Sessão no portal	Cookie `httpOnly`; token do cliente em memória	XSS não lê o token; `localStorage` persiste entre abas.
Contrato	`packages/contracts` com `implements`	Mudança de formato quebra a compilação do portal.
Configuração	`app.setup.ts` compartilhado	Dev e e2e usam o mesmo pipeline — sem divergência.

TEMA	DECISÃO	POR QUÊ
Seed	Somente por comando explícito	O banco nasce vazio; o que entra é sempre uma decisão consciente.
Migrations	Serviço separado no Compose	O CLI do Prisma fica fora da imagem final.
Cobertura	Piso configurado	A suíte falha se a qualidade cair.
Monorepo	npm workspaces, sem Turborepo/Nx	Com dois apps, os scripts da raiz bastam — menos uma camada de abstração.

12. Limitações conhecidas e próximos passos

12.1 Limitações assumidas

ITEM	SITUAÇÃO
Rate limit distribuído	O limitador é em memória: com várias instâncias da API, o limite vale por instância. Redis resolveria.
Refresh por dispositivo	A rotação é por sessão; falta listar e revogar os dispositivos conectados.
Histórico de itens	Ao trocar a lista de itens, os antigos são removidos fisicamente. Com histórico, também teriam `deleted_at`.
Notificações	A mudança de status é síncrona; com e-mail ou SMS, viraria um job.
Observabilidade	Sem tracing distribuído nem métricas de negócio.
Testes de carga	Há teste de concorrência; falta uma suíte com k6 ou autocannon.
CI/CD	Adiado por decisão — os scripts já existem na raiz do monorepo.
Testes e2e do portal	Sem Playwright; o portal é exercitado manualmente e via API.

12.2 Próximos passos sugeridos

1. **CI no GitHub Actions** — `lint → typecheck → test → build`; os scripts já existem.
2. **Testes e2e do portal com Playwright** — cobrindo login, criação de pedido e mudança de status pela interface.
3. **Rate limit com Redis** — para rodar múltiplas instâncias da API atrás de um balanceador.
4. **Refresh tokens por dispositivo** — listar e revogar sessões ativas.
5. **Histórico de itens** — preservar a auditoria de troca de itens.
6. **Fila de notificações** — mudança de status publica um evento.
7. **Observabilidade** — tracing (OpenTelemetry) e métricas de negócio.

13. Anexo: referência de comandos

13.1 Subir o ambiente

```
# Stack completa (banco + migrations + API + portal)
cp .env.example .env
docker compose up -d --build

# Portal      http://localhost:3001
# API         http://localhost:3000/api/v1
# Swagger UI  http://localhost:3000/docs

# Só o banco de desenvolvimento
docker compose up -d db

# Banco de testes (profile)
docker compose --profile test up -d db-test
```

13.2 Desenvolvimento local

```
npm install                # postinstall já gera o Prisma Client
npm run prisma:migrate:dev   # aplica migrations no banco de dev
npm run dev:api              # API em watch (3000)
npm run dev:web              # portal em dev (3001)
```

13.3 Dados de demonstração

```
npm run seed        # popula direto no Postgres (idempotente)
npm run seed:api    # popula passando pela API (a API precisa estar no ar)
npm run db:reset    # recria o schema e roda a seed
```

13.4 Qualidade

```
npm run lint        # ESLint (raiz + portal)
npm run format      # Prettier
npm run typecheck   # type-check dos três workspaces
npm test            # 64 testes unitários
npm run test:e2e    # 89 testes e2e
npm run test:all    # as duas suítes
npm run test:cov    # cobertura da suíte unitária
npm run test:e2e:cov  # cobertura da suíte e2e (com piso)
```

13.5 Shell dos exemplos

Os comandos desta seção usam **bash/zsh**. No **fish**, a atribuição de variável tem outra sintaxe — troque `TOKEN=$(...)` por `set TOKEN (...)`. O restante (comandos, `&&`, continuação de linha) é igual nos dois.

13.6 Credenciais da seed

E-MAIL	SENHA	PAPEL
`admin@naporta.local`	`Admin@1234`	`ADMIN` — vê todos os pedidos
`operador@naporta.local`	`Operador@1234`	`USER` — só os próprios pedidos
`ana@naporta.local`	`Cliente@1234`	`USER` — só os próprios pedidos

Exemplo: criando um pedido pela API

```
TOKEN=$(curl -s -X POST http://localhost:3000/api/v1/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"admin@naporta.local","password":"Admin@1234"}' | jq -r .accessToken)

curl -s -X POST http://localhost:3000/api/v1/orders \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d '{"expectedDeliveryDate":"2026-12-25","customerName":"Maria Silva",
       "customerDocument":"11144477735",
       "address":{"street":"Rua das Flores","number":"123","neighborhood":"Centro",
                  "city":"São Paulo","state":"SP","zipCode":"01001000"},
       "items":[{"description":"Caixa de papelão","quantity":2,"unitPrice":49.90}]}' | jq
```